BLACK PEN Answer all questions first. **GREEN PEN** Then: tick and rewrite mistakes.

LESSON 012 · Scope & parameter passing in C#

Name: _____ Class: _____ Date: _____

Work through the pages in order. Answer in BLACK PEN first. Check in GREEN PEN only after the Exit Ticket.

Do Now · Recall previous learning

Q1. What is the difference between a C# function and a procedure-style method?

Q2. Which C# loop always executes its body at least once?

Q3. What does == do in a C# condition?

Q4. In Triple(7) , identify the argument. In Triple(int number) , identify the parameter.

BLACK PEN Answer all questions first. **GREEN PEN** Then: tick and rewrite mistakes.

I Do · Read new knowledge

Key vocabulary

Recall the term — Scope

Recall the term — Local variable

Recall the term — Global variable

Recall the term — Parameter

Recall the term — Argument

Recall the term — Pass by value

Recall the term — Pass by reference

New learning

Scope controls where a variable can be used — A local variable is declared inside a method or block and can only be used within that scope. — For this course, we use a class-level static field to demonstrate the OCR idea of a global variable that can be accessed by multiple methods.

Parameters are local to the method — A parameter receives data when a method is called and behaves as a local variable inside that method. — CODE:

```
public static int Triple(int number)
{
return number * 3;
}
```

By value = the method receives a copy — CODE:

```
public static void Change(int number)
{
number = 100;
```

} — Changing number changes only the method's local copy. The original variable in Main() is unchanged.

ref gives access to the original variable — CODE:

```
public static void Change(ref int number)
{
number = 100;
}

// call
```

Change(ref value); — Write ref in both places: in the method parameter and in the method call.

BLACK PEN Answer all questions first. **GREEN PEN** Then: tick and rewrite mistakes.

Worked example

Why doesn't the original value change?

1. Start in Main — CODE:
int score = 10; — The variable score belongs to Main() and currently stores 10.

2. The method receives a copy — CODE:
public static void ChangeByValue(int number)
{
number = 100;
} — The parameter number is a separate local variable initialised with the value 10.

3. Only number becomes 100 — CODE:
ChangeByValue(score);

inside method:
number = 100

back in Main:
score = 10 — When the method ends, its local parameter is no longer needed.

4. Reference passing changes the original — CODE:
ChangeByReference(ref score);

// method
public static void ChangeByReference(ref int number)
{
number = 100;
}

We Do · Practise with guidance

Identify the scope

```
public static int total = 0;

public static void AddOne()
{
    int amount = 1;
    total = total + amount;
}
```

Which variable can be accessed by multiple methods? Which is local to AddOne() ?

Classify each variable

BLACK PEN Answer all questions first. **GREEN PEN** Then: tick and rewrite mistakes.

Predict pass-by-value

```
int x = 5;
DoubleValue(x);
Console.WriteLine(x);

public static void DoubleValue(int number)
{
    number = number * 2;
}
```

What is output?

Track the two variables

number is a local copy, so changing it does not change x .

Make the change persist

Modify the previous example so x becomes 10 after the call.

Where must ref appear?

Using ref in both places lets the method change the original variable.

Explain by value vs by reference

OCR expects more than the words “copy” and “address”. Explain what happens to the original data .

Complete both explanations

BLACK PEN Answer all questions first. **GREEN PEN** Then: tick and rewrite mistakes.

You Do · Work independently

BLACK PEN: Write your own answers. Do not look at the Review & Improve pages yet.

Question 1

Local variables in two methods
Create two procedure-style methods: First() and Second() .
Inside each method declare a local variable called result , but give it a different value.
Output each value.

Question 2

Prove pass-by-value
Set int score = 20; in Main() .
Create SetToZero(int number) that changes number to 0.
Call the method, then output score .

Question 3

Change the original with ref
Adapt Challenge 2 so the original score becomes 0.
Rename the method SetToZeroByReference and use ref correctly in both required places.

Question 4

Swap two values
Create Swap(ref int first, ref int second) .
Use one local temporary variable so that the two original values exchange places.
In Main() , start with a = 10 and b = 25 , call the method and output both values.

BLACK PEN Answer all questions first. **GREEN PEN** Then: tick and rewrite mistakes.

Exit Ticket · Answer before checking

Passing an integer parameter by value means changing the parameter does not change the original integer variable.

BLACK PEN Answer all questions first. **GREEN PEN** Then: tick and rewrite mistakes.

Review & Improve · GREEN PEN

Now change to GREEN PEN. Find each original question. Tick correct answers; read the model and rewrite mistakes.

Page 1 onwards · Do Now

Q1. A function returns a value . A procedure-style method uses void and does not return a value.

Q2. do...while Its condition is checked after the body has run.

Q3. It compares two values for equality . = assigns; == compares.

Q4. Argument = 7 · parameter = number The argument is the actual value supplied; the parameter is the named input in the method definition.

You Do pages · Independent questions

Q1. Local variables in two methods
CODE:

```
using System;

public class Program
{
public static void Main()
{
First();
Second();
}

public static void First()
{
int result = 10;
Console.WriteLine(result);
}

public static void Second()
{
int result = 25;
Console.WriteLine(result);
}
}
```

Why it works: each result variable is local to its own method, so the identifiers do not conflict.

Q2. Prove pass-by-value
CODE:

```
using System;

public class Program
{
public static void Main()
{
int score = 20;

SetToZero(score);

Console.WriteLine(score);
}

public static void SetToZero(int number)
{
number = 0;
}
}
```

Why it works: number receives a copy of the value. Changing that local copy does not change score .

Q3. Change the original with ref
CODE:

```
using System;

public class Program
{
public static void Main()
{
int score = 20;

Console.WriteLine("Before: " + score);
SetToZeroByReference(ref score);
Console.WriteLine("After: " + score);
```

```
}

public static void SetToZeroByReference(ref int number)
{
number = 0;
}
}
```

Key syntax: ref appears in both the method parameter and the method call.

Q4. Swap two values

CODE:

```
using System;

public class Program
{
public static void Main()
{
int a = 10;
int b = 25;

Swap(ref a, ref b);

Console.WriteLine("a = " + a);
Console.WriteLine("b = " + b);
}

public static void Swap(ref int first, ref int second)
{
int temporary = first;
first = second;
second = temporary;
}
}
```

Why it works: both parameters refer to the original variables. The local temporary variable prevents one original value being lost during the swap.

Exit Ticket · Model answer

True. For an integer passed by value in C#, the called method receives a copy.

BLACK PEN Answer all questions first. **GREEN PEN** Then: tick and rewrite mistakes.

Try the question again

Choose one question you got wrong. All correct? Choose the hardest.

1. Copy the whole original question:

2. Write a complete new answer in GREEN PEN. Show working if needed: